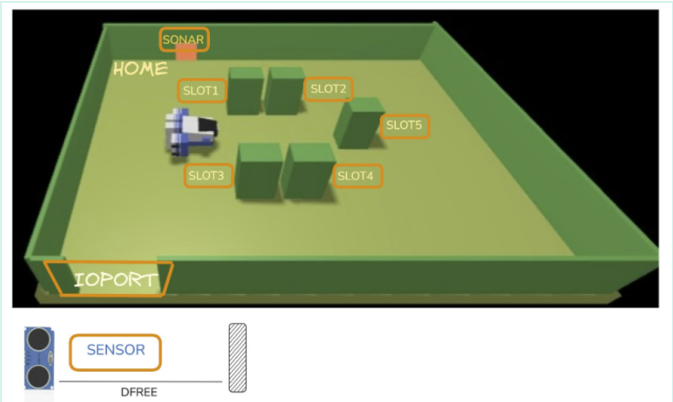
CargoBot26: SPRINT1

Introduction

GOAL:
- Analizzare e sviluppare i componenti cargoservice e cargorobot e le loro logiche

Requirements



In the picture above:

• The **slots1-4** depict the hold areas reserved to store one *container* each,

• The **slot5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

• It sends the answer retrylater, if the **IOPort** is currently occupied by a container or if the system is *Out of service*

• It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.

• Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the **slot5** (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

• the current state of the *hold*

• the message **'Service working'**, when it is all is going well

• the message **'Out of service'** if the *sonar sensor* measures a distance D > DFREE for at least 3 secs (perhaps a failure of the *sonar*).

Requirement Analysis

A partire da quanto svolto nello Sprint0, ci concentriamo sui seguenti requisiti:

• Studio dei componenti non-attivi del sistema e loro integrazione.

• Studio del macro-componente **cargoservice** e della logica di business relativa alle richieste di movimentazione e allocazione dei container.

• Studio del macro-componente **cargorobot**; il quale realizza le operazioni relative al DDR.

Problem Analysis

Componenti non-attivi e introduzione di cargohold:

• Per rappresentare in modo strutturato lo stato della hold e dei suoi componenti, si introducono apposite classi POJO (già presentate e discusse nello Sprint0), che consentono di modellare le informazioni relative alla configurazione spaziale della stiva e allo stato degli slot.

• Una volta definita la rappresentazione della hold e dei relativi componenti, è necessario individuare quale entità del sistema debba essere responsabile della gestione e della persistenza del suo stato durante l'esecuzione. Tale responsabilità non dovrebbe essere affidata a cargoservice, poiché un suo eventuale riavvio a runtime comporterebbe la nuova istanziazione degli oggetti utilizzati per rappresentare lo stato della stiva, con il rischio di perdere le informazioni accumulate durante l'esecuzione. Per evitare questa dipendenza e garantire una maggiore separazione delle responsabilità, si sceglie di introdurre un **nuovo microservizio proattivo**, dedicato alla gestione e al mantenimento dello stato della hold e dei suoi componenti: **cargohold**. Questa entità avrà il compito di monitorare e aggiornare lo stato del sistema, rendendolo disponibile agli altri servizi attraverso opportune interfacce di comunicazione. In questo modo, cargoservice potrà concentrarsi sulle proprie responsabilità applicative, delegando la gestione dello stato a un componente dedicato.

• Le modalità di mantenimento dello stato sono affidate al progettista. Si individuano, tuttavia, tre possibili soluzioni:
- **Salvataggio su database**: lo stato del sistema viene salvato e mantenuto in un database.
  (Questa soluzione non è ritenuta idonea dal committente, in quanto comporterebbe la necessità di gestire e mantenere un database dedicato.)
- **Salvataggio su file locale**: lo stato del sistema viene salvato in un semplice file sullo stesso nodo di elaborazione.
- **Gestione a runtime**: lo stato del sistema dipende dal processo in esecuzione. In caso di errore o arresto di cargohold, il sistema viene reinizializzato.

• L'introduzione di **cargohold** come servizio separato rende la sua gestione scalabile e modificabile nel tempo.

Relazione cargorobot ↔ robotsmart26:

• Sulla base dei requisiti e delle indicazioni fornite dal committente, si stabilisce che i due componenti software robotsmart26 e cargorobot siano da considerarsi equivalenti e facciano riferimento alla medesima entità all'interno dell'architettura del sistema. Tale scelta consente di uniformare la terminologia utilizzata nella documentazione, evitando ambiguità nell'identificazione del componente e nella descrizione delle sue responsabilità. Di conseguenza, da questo punto in avanti, **i termini robotsmart26 e cargorobot saranno utilizzati in modo intercambiabile**, senza che ciò implichi l'esistenza di due componenti software distinti.

• Si osserva che robotsmart26 non gestisce lo stato degli slot, ma come già anticipato, sarà compito cargohold.

Definizione dei QActor e dei contesti:

Di seguito definiamo gli attori in gioco modellando un primo sistema:

```
System cargocompany

Context ctxcargocompany ip [ host="localhost" port=8040]
Context ctxrobotsmart ip [host="127.0.0.1" port=8050 ]

//...

ExternalQActor robotsmart context ctxrobotsmart

//...

QActor cargohold context ctxcargocompany

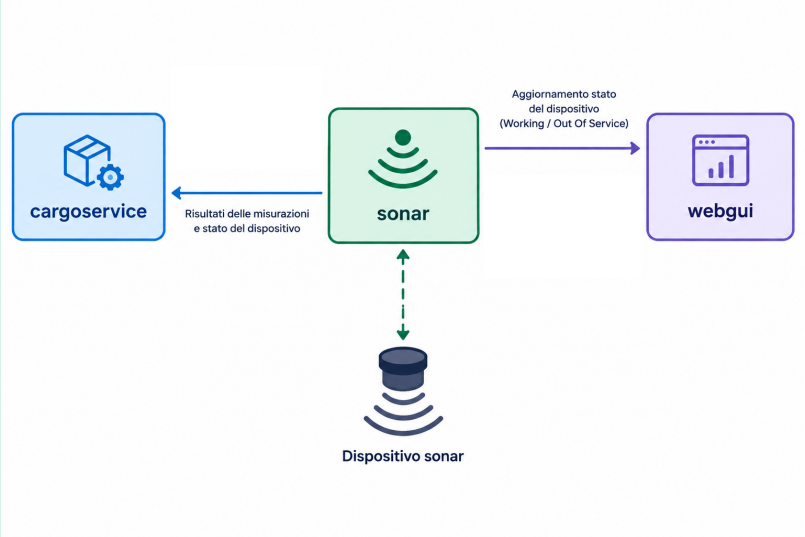
//...

QActor cargoservice context ctxcargocompany
```

Relazione con il PicoW (sonar - led):

• La logica applicativa relativa alle misurazioni del sonar, definita dai requisiti, è interamente delegata al QActor dedicato, denominato **sonar**. Questo si occupa di elaborare le misurazioni e di fornire agli attori interessati del sistema esclusivamente le informazioni essenziali per il suo corretto funzionamento. Si evidenzia, in questo modo, una netta separazione dei compiti e una chiara distribuzione delle responsabilità tra i diversi componenti del sistema.

• Dai requisiti emerge inoltre che un entità dovrà interagire con la **webgui** per segnalare lo stato operativo del dispositivo hardware, indicando se si trova in condizione di Out Of Service oppure Service Working. Questo rappresenta un'ulteriore motivazione a favore della scelta di delegare la logica relativa alle misurazioni al servizio **sonar**, centralizzando in un unico componente le responsabilità legate alla gestione del dispositivo.



• Si individuano le informazioni di interesse di **cargoservice**:
- Posizionamento di un container nello zona di carico
- Il sonar è Out Of Service

• Il **led** collegato al PicoW deve rispecchiare lo stato del sistema, accendendosi durante la fase di engaged e spegnendosi in quella di disengaged. A tale scopo, cargoservice comunica con il relativo attore responsabile servendosi di un messaggio dispatch.

Dettaglio messaggi:

• **cargohold** comunica tramite la seguente sintassi:

• **cargohold ↔ cargoservice**

```
Request requestSlot : requestSlot(NOARGS)
Reply slotReserved : slotReserved(ANSWER) for requestSlot
Reply slotFailed : slotFailed(REASON) for requestSlot

Request setSlot : setSlot(SLOTID, STATE)
Reply setSlotChanged : setSlotChanged(NOARGS) for setSlot
Reply setSlotFailed : setSlotFailed(REASON) for setSlot

Request requestPosition : requestPosition(POSITION)
Reply slotPosition : slotPosition(X, Y) for requestPosition
Reply positionFailed : positionFailed(REASON) for requestPosition
```

Richieste attese:
- **setSlot**: slotId, stato "free", "reserved" o "occupied"
- **requestPosition**: slotId, "ioport" oppure "home"

Risposte attese:
- **slotReserved**: slotId riservato
- **slotPosition**: coordinate X, Y
- **slotFailed**: motivazione
- **setSlotFailed**: motivazione
- **positionFailed**: motivazione

• **cargoservice** comunica tramite la seguente sintassi:

• **cargoservice ↔ webgui**

```
Request requestToLoad : requestToLoad(NOARGS)
Reply loadAccepted : loadAccepted(ANSWER) for requestToLoad
Reply loadRejected : loadRejected(ANSWER) for requestToLoad
Reply loadRetryLater : loadRetryLater(ANSWER) for requestToLoad
```

Risposte attese:
- **loadAccepted**: slotId riservato
- **loadRejected**: "rejected"
- **loadRetryLater**: "retrylater"

• **cargoservice ↔ sonar**

```
Dispatch containerArrived : containerArrived(NOARGS)
Dispatch outOfService : outOfService(NOARGS)
```

• **cargoservice ↔ led**

```
Dispatch ledState  : ledState(STATE)
```

Messaggi attesi:
- **ledState**: "on", "off"

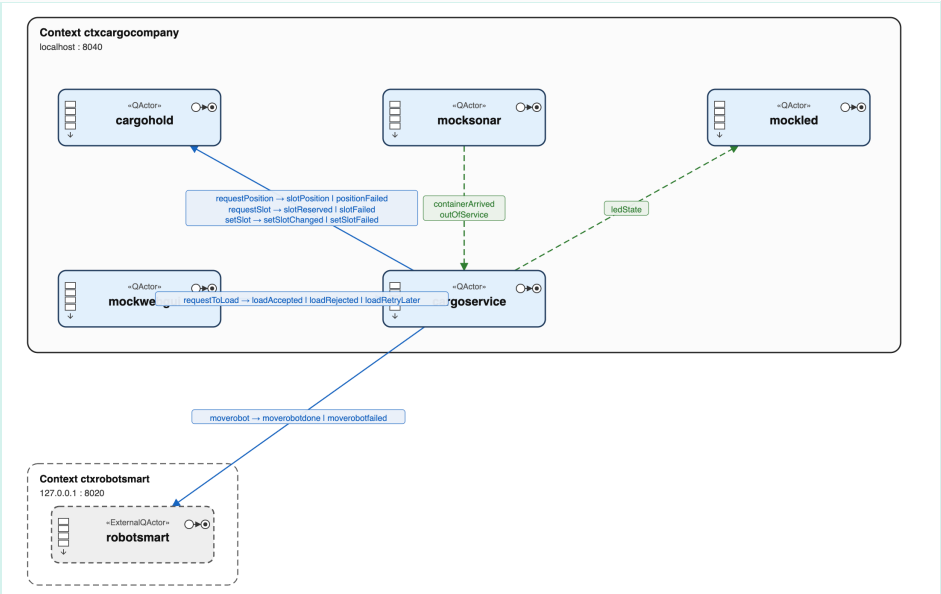
• **cargoservice** coordina i movimenti del **cargorobot** e si interfaccia con il software del committente, adattandosi ai suoi formati di messaggistica. Si evidenziano i più rilevanti:

• **cargoservice ↔ cargorobot (robotsmart26)**

```
Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
Reply moverobotdone : moverobotdone(ARG) for moverobot
Reply moverobotfailed : moverobotfailed(PLANDONE, PLANTODO) for moverobot
```

• Notiamo gli attori, essendo configurati come un QActor (Qak), presentano una propria **coda locale** in cui sono depositati i messaggi a loro inviati da altri attori. Per capirne bene le conseguenza è possibile affidarsi alla documentazione Qak.

Modello:



Test plans

• Vengono definiti test che permettono di esplicitare il comportamento atteso di **cargoservice** in relazione ai messaggi.
Di seguito vengono presentati degli esempi esemplificativi:

```
@Test
public void testRequestToLoadAccepted() throws Exception {
    String req = CommUtils.buildRequest(
            "mock",
            "requestToLoad",
            "requestToLoad(NOARGS)",
            "cargoservice"
    ).toString();

    System.out.println("Richiesta: " + req);

    String response = conn.request(req);

    System.out.println("Risposta: " + response);

    assertTrue("La risposta deve contenere slot",
            response.contains("slot"));
    assertFalse("La richiesta non dovrebbe essere rifiutata",
            response.contains("rejected"));
}

@Test
public void testRequestToLoadRejected() throws Exception {
    String req = CommUtils.buildRequest(
            "mock",
            "requestToLoad",
            "requestToLoad(NOARGS)",
            "cargoservice"
    ).toString();

    System.out.println("Richiesta: " + req);

    String response = conn.request(req);

    System.out.println("Risposta: " + response);

    assertTrue("La risposta deve essere rifiutata",
            response.contains("rejected"));
    assertFalse("La richiesta non dovrebbe specificare uno slot",
            response.contains("slot"));
}
```

Project

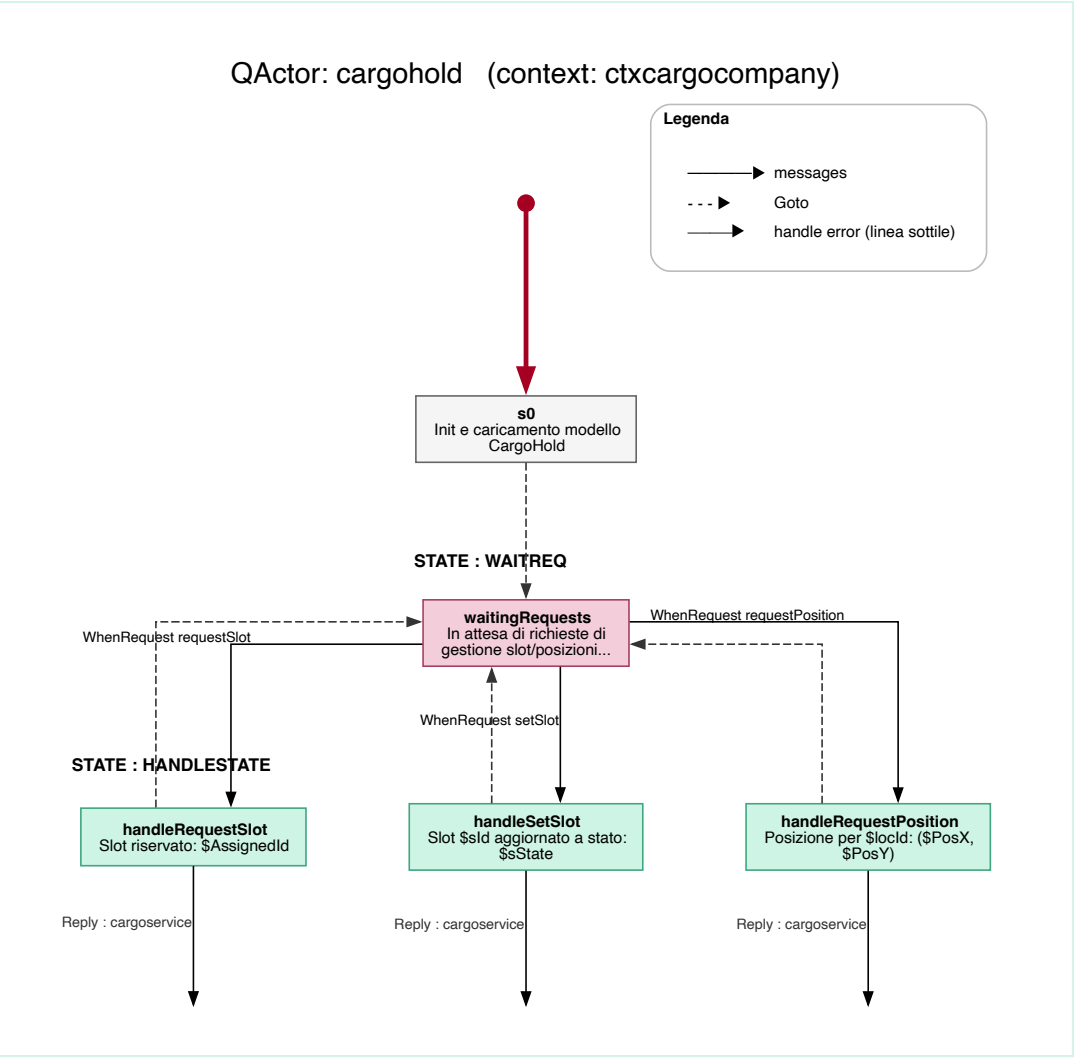
cargoHold-1.0.jar

• Sulla base delle interfacce individuate nello Sprint0, in relazione ai componenti non-attivi,
viene prodotta una libreria contente l'insieme delle classi POJO: **cargoHold-1.0.jar**.

- **Position**
- **Slot**
- **Hold**

• Il servizio **cargohold** usa **cargoHold-1.0.jar**.
Attualmente non si ritiene necessario il salvataggio dello stato del sistema su un file esterno.
Lo stato verrà mantenuto finchè il servizio rimarrà attivo sul nodo.

Stati del servizio cargohold:



• MARCO-STATO: WAITREQ

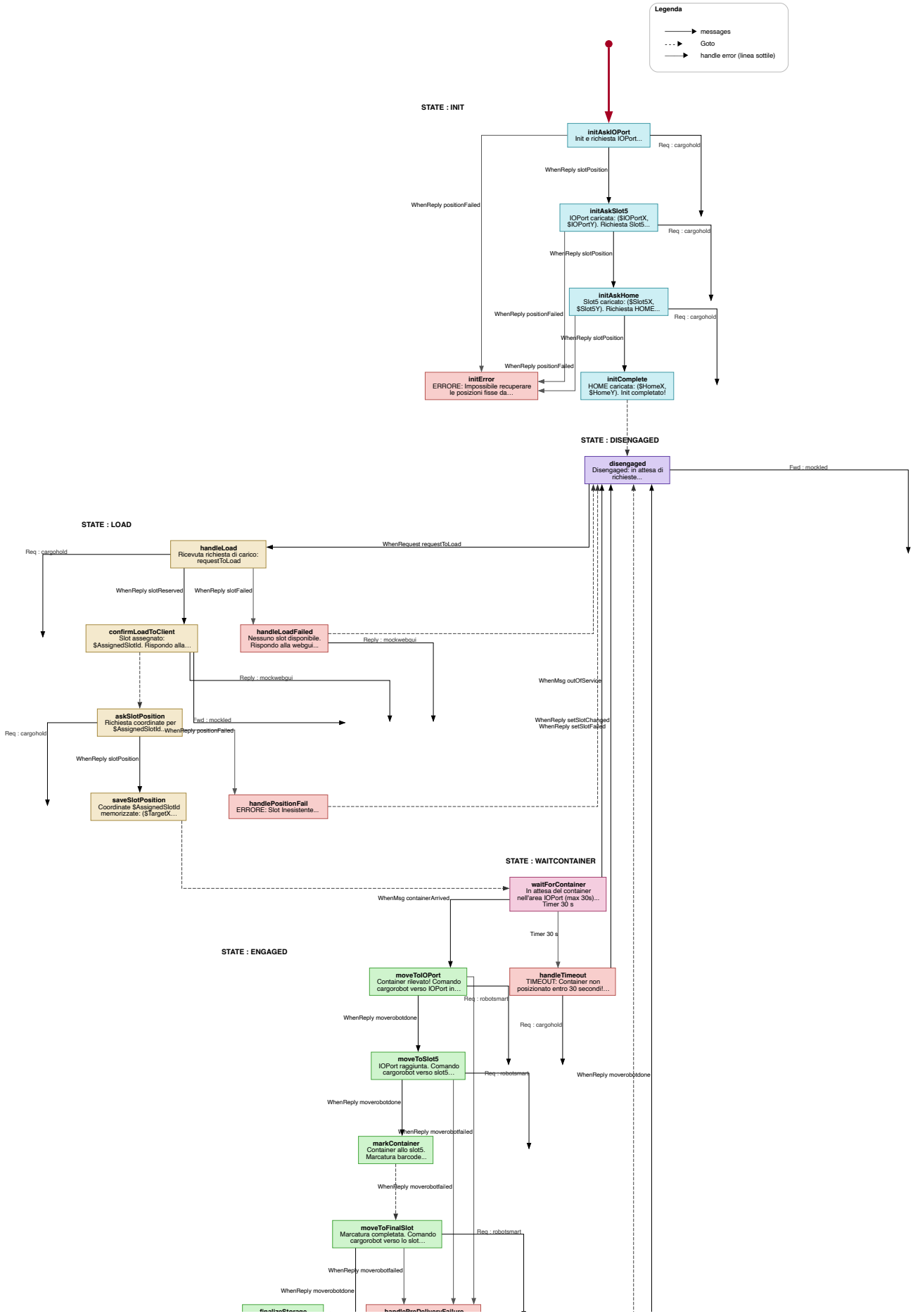
Stato	Descrizione
waitingRequest	In attesa di richieste per la gestione della stiva.

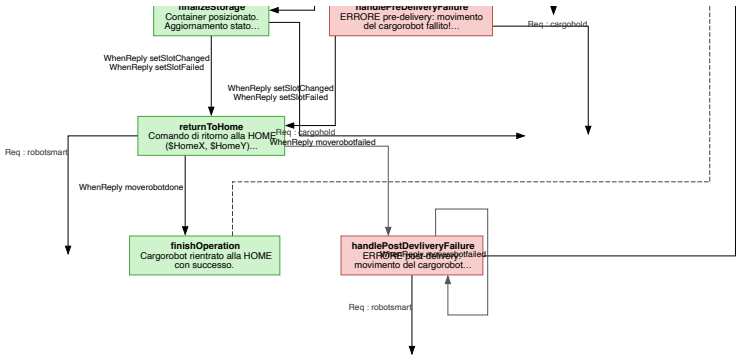
• MARCO-STATO: HANDLESTATE

Stato	Descrizione
handleRequestSlot	Verifica la presenza di uno slot libero.
handleSetSlot	Cambia lo stato di uno slot.
handleRequestPosition	Ritorna la posizione di un elemento del sistema.

Stati del servizio cargoservice:

QActor: cargoservice (context: ctxcargocompany)





• **MARCO-STATO: INIT**

Stato	Descrizione
initAskIOPort	Stato iniziale del sistema. Richiede a cargohold le coordinate dell'IOPort.
initAskSlot5	Salva le coordinate dell'IOPort. Richiede a cargohold le coordinate dello Slot5.
initAskHome	Salva le coordinate dello Slot5. Richiede a cargohold le coordinate della Home.
initComplete	Salva le coordinate della Home. Termina la fase di INIT.
initError	Gestisce eventuali errori durante l'inizializzazione del sistema, per i quali è necessario procedere al riavvio.

• **MARCO-STATO: DISENGAGED**

Stato	Descrizione
disengaged	Stato in cui il sistema rimane in attesa di nuove richieste di carico.

• **MARCO-STATO: LOAD**

Stato	Descrizione
handleLoad	Gestisce una richiesta di carico. Richiede a cargohold se è presente uno slot libero.
confirmLoadToClient	Se lo slot è stato riservarto, comunica lo slotId alla webgui.
handleLoadFailed	Se non sono presenti slot liberi, comunica 'retrylater' alla webgui.
askSlotPosition	Richiede a cargohold le coordinate dello slot riservato.
saveSlotPosition	Salva le coordinate dello slot riservarto.

• **MARCO-STATO: WAITCONTAINER**

Stato	Descrizione
waitForContainer	Attende (max 30 sec.) che il sonar rilevi la presenza di un container nella zona di carico IOPort.
handleTimeOut	Lo slot precedentemente assegnato torna libero.

• **MARCO-STATO: ENGAGED**

Stato	Descrizione
moveToIOPort	Il DDR viene comandato fino all'IOPOrt.
moveSlot5	Il DDR viene comandato fino allo Slot5.
markContainer	Il DDR attende la marcatura del container.
moveToFinalSlot	Il DDR viene comandato fino allo slot riservato.
finalizeStorage	Se tutto è andato a buon fine, cargoservice chiede a cargohold di impostare lo stato dello slot come occupato.
returnToHome	Il DDR ritorna alla Home.
finishOperation	Il DDR ha finito di svolgere il suo ciclo di operazioni ed è tornato con successo alla Home.

Stato	Descrizione
handlePreDeliveryFailure	Gestisce errori di movimento legati ad alcuni stati (come da immagine).
handlePostDeliveryFailure	Gestisce errori di movimento legati ad alcuni stati (come da immagine).

Modello:

• **Qak eseguibile** del sistema.

Testing

Piani di test sviluppati:

• **Codice test cargohold**
• **Codice test cargoservice**

Team

Alessandro Quaranta - alessandro.quaranta@studio.unibo.it
Gaia Verzelli - gaia.verzelli@studio.unibo.it

Project GitHub repository:
https://github.com/bbnogaia/cargo-robot-2026

Student GitHub repository:
https://github.com/alequaranta/iss2026quaranta